



Tehnologii WEB

Curs 7

JavaScript

Cod Teams: **6gyjd1x**

Anul III, semestrul II

2026

JavaScript - continuare

JavaScript este un limbaj de programare **interpretat, dinamic, orientat pe obiecte**, folosit în principal în browser pentru a controla comportamentul paginilor web.

🔧 Caracteristici fundamentale

Limbaj single-threaded → rulează pe un singur fir de execuție.

Event-driven → reacționează la evenimente (click, tastare, încărcare).

Non-blocking → folosește asincronism pentru a evita blocarea execuției.

Tipare dinamice → variabilele își pot schimba tipul în timpul rulării.

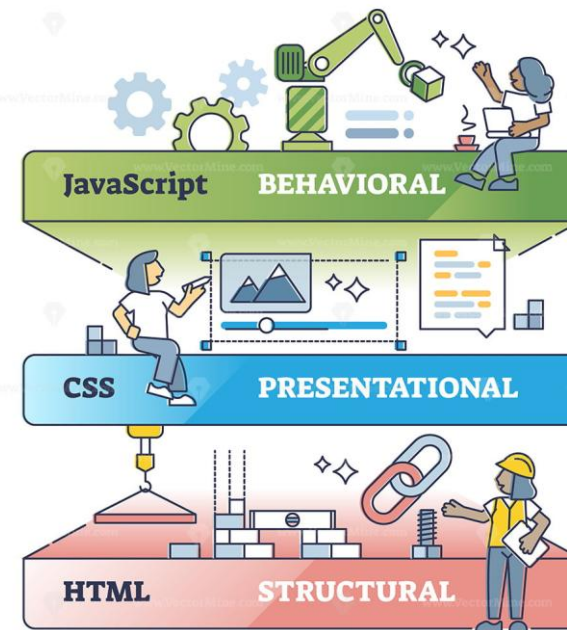
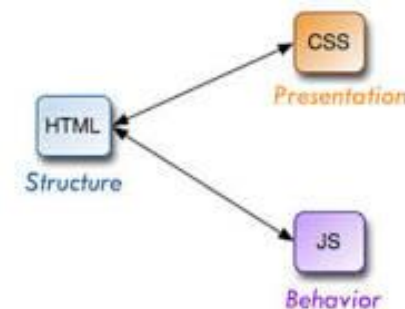
Prototype-based → obiectele moștenesc prin prototip, nu prin clase tradiționale.

🔗 Rol în ecosistemul Web:

HTML → structură

CSS → stil

JavaScript → logică, interactivitate, dinamism



DOM: Document Object Model

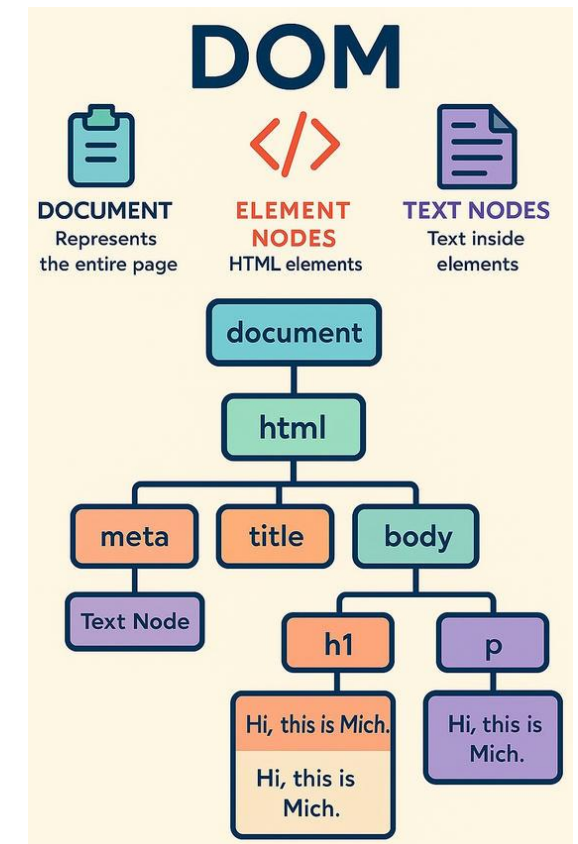
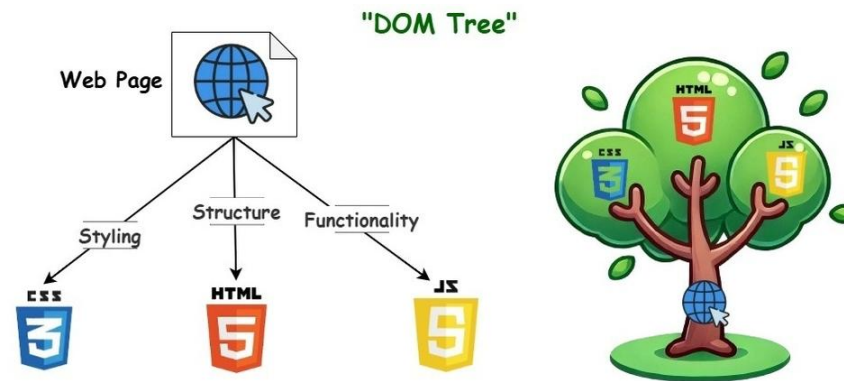
DOM este o **reprezentare internă** a paginii web sub forma unui arbore de noduri. Browserul transformă codul HTML într-o structură de obiecte accesibile prin JavaScript.

Tipuri de noduri:

- **Element nodes** (ex: <div>)
- **Text nodes** (conținutul dintre tag-uri)
- **Attribute nodes** (atribute HTML)
- **Document node** (rădăcina arborelui)

De ce este important?

- Permite manipularea dinamică a paginii.
- Este baza interactivității în web.



Selectarea elementelor

Metode moderne:

- `document.querySelector()` → selectează primul element care corespunde unui selector CSS.
- `document.querySelectorAll()` → selectează toate elementele.

Metode clasice:

- `getElementById()`
- `getElementsByClassName()`
- `getElementsByTagName()`

Rezultat:

- `titlu` → obiect DOM pentru elementul cu id-ul „titlu”
- butoane → listă statică de elemente
- `HTMLCollection` = live (se actualizează automat)
- `NodeList` = static

```
HTML
2 * <head>
3   <meta charset="UTF-8">
4   <title>Selectare DOM</title>
5   <link rel="stylesheet" href="style.css">
6 </head>
7 * <body>
8 * <h1 id="titlu">Titlu principal</h1>
9 * <p class="text">Paragraf 1</p>
10 * <p class="text">Paragraf 2</p>
11 * <div class="box">DIV 1</div>
12 * <div class="box">DIV 2</div>
13 * <button id="btn">Apasă-mă</button>
14   <script src="script.js"></script>
15 </body>
16 </html>
```

Titlu principal

Paragraf 1

Paragraf 2

DIV 1

DIV 2

Apasă-mă

Titlu principal

Paragraf 1

Paragraf 2

DIV 1

DIV 2

Apasă-mă

```
JS
1 // 1. METODE CLASICE
2 // getElementById → selectează UN element după id
3 let titlu = document.getElementById("titlu");
4 // modificăm textul
5 titlu.style.color = "blue";
6 // getElementsByClassName → selectează toate elementele cu o clasă
7 let paragrafe = document.getElementsByClassName("text");
8 // HTMLCollection → trebuie parcurs
9 for (let i = 0; i < paragrafe.length; i++) {
10   paragrafe[i].style.color = "green";
11 }
12 // getElementsByTagName → selectează după tag (ex: div)
13 let divuri1 = document.getElementsByTagName("div1");
14 for (let i = 0; i < divuri1.length; i++) {
15   divuri1[i].style.border = "2px solid black";
16 }
17 // selectăm toate div-urile cu clasa "box"
18 let divuri = document.querySelectorAll(".box");
19 // selectăm butonul
20 let btn = document.getElementById("btn");
21 // adăugăm eveniment de click
22 btn.addEventListener("click", () => {
23   // pentru fiecare div
24   divuri.forEach(div => {
25     // adăugăm sau scoatem clasa "activ"
26     div.classList.toggle("activ");
27   });
28 });
```

CSS

```
1 * body {
2   font-family: Arial;
3   padding: 30px;
4 }
5 * .box {
6   padding: 10px;
7   margin: 5px;
8   background-color: lightgray;
9 }
10 * .activ {
11   background-color: yellow;
12 }
```

Evenimente în JavaScript

Ce este un eveniment?

O acțiune detectată de browser:

- click
- mouseover
- keydown
- submit
- load

Cum funcționează?

JavaScript folosește un Event Loop care ascultă evenimente și execută funcțiile asociate.

```
HTML
1 * <html lang="ro">
2 * <head>
3   <meta charset="UTF-8">
4 *   <title>Evenimente JavaScript</title>
5 *   <style>
6 *     body {
7       font-family: Arial;
8       padding: 20px;
9     }
10 *    .box {
11      width: 200px;
12      height: 100px;
13      background-color: lightgray;
14      text-align: center;
15      line-height: 100px;
16      margin-bottom: 10px;
17    }
18 *    .hover {
19      background-color: orange;
20    }
21 *    .click {
22      background-color: green;
23      color: white;
24    }
25 *    input {
26      padding: 5px;
27      margin-bottom: 10px;
28    }
29 * </style></head><body>
30 * <h2>Exemple Evenimente</h2>
```

```
HTML
30 * <!-- CLICK -->
31 * <button id="btn">Click pe mine</button>
32 * <!-- MOUSEOVER -->
33 * <div class="box" id="hoverBox">Treci peste mine</div>
34 * <!-- KEYDOWN -->
35 * <input type="text" id="inputText" placeholder="Tastează ceva...">
36 * <p id="output"></p>
37 * <!-- SUBMIT -->
38 * <form id="form">
39 *   <input type="text" placeholder="Nume" required>
40 *   <button type="submit">Trimite</button>
41 * </form>
42 * <script>
43 * // LOAD (când pagina se încarcă)
44 * window.addEventListener("load", () => {
45 *   console.log("Pagina s-a încărcat complet");
46 * });
47 * // CLICK
48 * let btn = document.getElementById("btn");
49 * btn.addEventListener("click", () => {
50 *   btn.classList.toggle("click");
51 *   btn.textContent = "Ai dat click!";
52 * });
53 * // MOUSEOVER
54 * let box = document.getElementById("hoverBox");
55 * box.addEventListener("mouseover", () => {
56 *   box.classList.add("hover");
57 * });
58 * box.addEventListener("mouseout", () => {
59 *   box.classList.remove("hover");
60 * });
61 * // KEYDOWN
62 * let input = document.getElementById("inputText");
63 * let output = document.getElementById("output");
64 * input.addEventListener("keydown", (e) => {
65 *   output.textContent = "Ai apăsat tasta: " + e.key;
66 * });
67 * // SUBMIT
68 * let form = document.getElementById("form");
69 * form.addEventListener("submit", (e) => {
70 *   e.preventDefault(); // oprește refresh-ul paginii
71 *   alert("Formular trimis!");
72 * });
73 * </script></body></html>
```

Exemple Evenimente

Click pe mine

Treci peste mine

Tastează ceva...

Nume

Trimite

Exemple Evenimente

Ai dat click!

Treci peste mine

Tastează ceva...

Nume

Trimite

Exemple Evenimente

Ai dat click!

Treci peste mine

M

Ai apăsât tasta: M

Nume

Trimite

Exemple Evenimente

Ai dat click!

Treci peste mine

Tehnologii Web

Ai apăsât tasta: b

Nume

Trimite

Formular trimis!

OK

Exemple Evenimente

Ai dat click!

Treci peste mine

Tehnologii Web

Ai apăsât tasta: b

Formular

Trimite

Propagarea evenimentelor

Două faze:

- **Capturing phase** (de sus în jos)
- **Bubbling phase** (de jos în sus)

Implicații:

- Evenimentele pot fi interceptate la diferite niveluri ale DOM-ului.
- `event.stopPropagation()` oprește propagarea.

Exemplu:

```
div.addEventListener("click", () => console.log("DIV"));
```

```
btn.addEventListener("click", () => console.log("BUTTON"));
```

Rezultat:

Click pe buton → se execută întâi handler-ul butonului, apoi al div-ului (bubbling).

HTML

```
1 * <html lang="ro">
2 * <head>
3   <meta charset="UTF-8">
4 *   <title>Capturing vs Bubbling</title>
5   <link rel="stylesheet" href="style.css">
6 * </head><body>
7 * <h2>Click pe elementul din interior</h2>
8 * <div id="outer">
9   OUTER
10 *   <div id="middle">
11     MIDDLE
12 *     <div id="inner">
13       INNER (click aici)
14     </div>   </div></div>
15 <script src="script.js"></script>
16 </body></html>
```

CSS

```
1 * div {
2   padding: 30px;
3   border: 2px solid black;
4   margin: 10px;}
5 * #outer {
6   background-color: lightblue;}
7 * #middle {
8   background-color: lightgreen;}
9 * #inner {
10  background-color: lightcoral;
11  cursor: pointer;
12 }
```

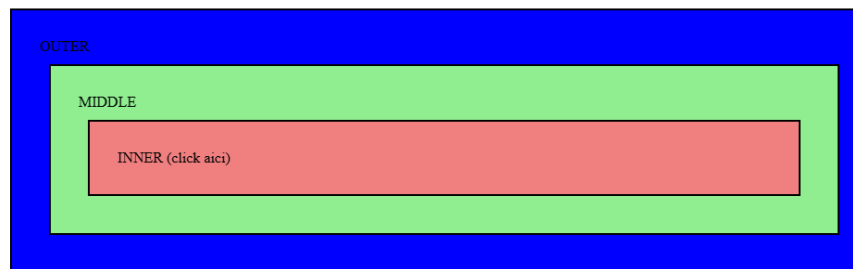
JS

```
1 // selectăm elementele
2 let outer = document.getElementById("outer");
3 let middle = document.getElementById("middle");
4 let inner = document.getElementById("inner");
5 // funcție ajutătoare pentru delay
6 * function schimbaCuloare(el, culoare, delay) {
7   *   setTimeout(() => {
8     *     el.style.backgroundColor = culoare;
9     *   }, delay);
10  * }
11 // CLICK PE INNER
12 * inner.addEventListener("click", () => {
13   *   // CAPTURING SIMULAT (sus -> jos)
14   *   schimbaCuloare(outer, "blue", 0);
15   *   schimbaCuloare(middle, "green", 500);
16   *   schimbaCuloare(inner, "red", 1000);
17   *   // BUBBLING (jos -> sus)
18   *   schimbaCuloare(inner, "lightcoral", 1500);
19   *   schimbaCuloare(middle, "lightgreen", 2000);
20   *   schimbaCuloare(outer, "lightblue", 2500);
21 * });
```

Click pe elementul din interior



Click pe elementul din interior



Click pe elementul din interior



Asincronism în JavaScript

De ce este necesar?

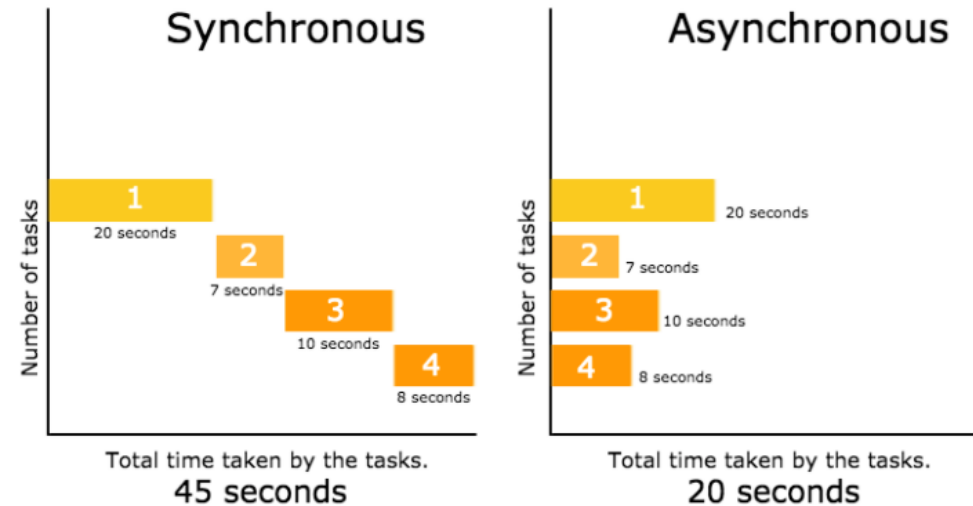
JavaScript rulează pe un singur fir → operațiile lente (rețea, timp de așteptare) ar bloca pagina.

Mecanisme:

- Callback-uri (model vechi)
- Promises (model modern)
- Async/Await (sintaxă simplificată pentru Promises)

Event Loop:

- Gestionează execuția codului
- Mută funcțiile în coada de execuție când sunt gata



```
HTML
1 * <html lang="ro">
2 * <head>
3   <meta charset="UTF-8">
4   <title>Asincronism JS</title>
5   <link rel="stylesheet"
6     href="style.css">
7 </head>
8 <body>
9 <h2>Asincronism în JavaScript</h2>
10 <button id="btn">Rulează exemplu</button>
11 <ul id="log"></ul>
12 <script src="script.js"></script>
13 </body>
14 </html>

CSS
1 * body {
2   font-family: Arial;
3   padding: 20px;
4 }
5 * button {
6   padding: 10px;
7   margin-bottom: 15px;
8 }
9 * li {
10   margin: 5px 0;
11 }
```

```
JS
1 let btn = document.getElementById("btn");
2 let log = document.getElementById("log");
3 // funcție pentru afișare în pagină
4 function afiseaza(text) {
5   let li =
6     document.createElement("li");
7   li.textContent = text;
8   log.appendChild(li);
9 }
10 btn.addEventListener("click", () => {
11   log.innerHTML = "";
12   afiseaza("1. Start cod (sincron)");
13   // CALLBACK (model vechi)
14   setTimeout(() => {
15     afiseaza("2. Callback executat
16       (după 2 secunde)");
17   }, 2000);
18   // PROMISE
19   let promisiune = new Promise((resolve,
20     reject) => {
21     setTimeout(() => {
22       resolve("3. Promise
23         rezolvat");
24     }, 3000);
25   });
26   promisiune.then(rez => {
27     afiseaza(rez);
28   });
29   // ASYNC / AWAIT
30   async function ruleazaAsync() {
31     let rezultat = await new
32       Promise(resolve => {
33         setTimeout(() => {
34           resolve("4. Async/Await
35             finalizat");
36         }, 4000);
37       });
38     afiseaza(rezultat);
39   }
40   ruleazaAsync();
41   // FINAL SINCRON
42   afiseaza("5. Final cod (sincron)");
43 });
```

Asincronism în JavaScript

Rulează exemplu

- 1. Start cod (sincron)
- 5. Final cod (sincron)

Asincronism în JavaScript

Rulează exemplu

- 1. Start cod (sincron)
- 5. Final cod (sincron)
- 2. Callback executat (după 2 secunde)

Asincronism în JavaScript

Rulează exemplu

- 1. Start cod (sincron)
- 5. Final cod (sincron)
- 2. Callback executat (după 2 secunde)
- 3. Promise rezolvat
- 4. Async/Await finalizat

Promises

Ce este o Promise?

Un obiect care reprezintă o valoare ce va fi disponibilă în viitor.

Stări:

- pending
- fulfilled
- rejected

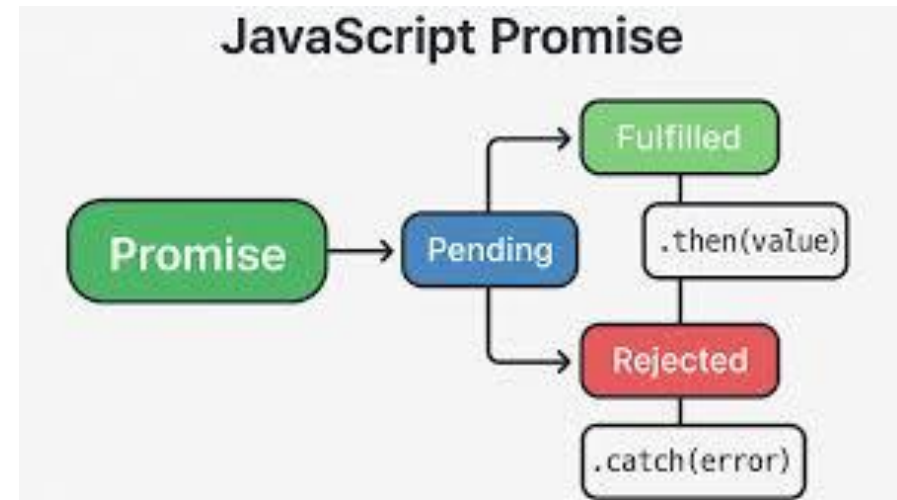
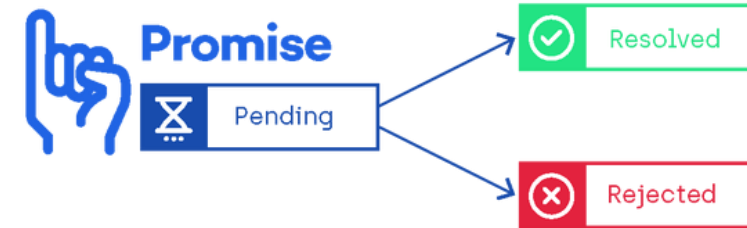
Exemplu:

```
const p = new Promise(resolve => {  
  setTimeout(() => resolve("Terminat!"), 1000);  
});
```

```
p.then(r => console.log(r));
```

Rezultat:

Terminat! (după 1 sec)



HTML

```
1 <html lang="ro">
2 <head>
3   <meta charset="UTF-8">
4   <title>Promise Example</title>
5   <link rel="stylesheet" href="style.css">
6 </head><body>
7 <h2>Exemplu Promise</h2>
8 <button id="successBtn">Succes</button>
9 <button id="errorBtn">Eroare</button>
10 <p id="status">Status: -</p>
11 <script src="script.js"></script>
12 </body></html>
```

* CSS

```
1 body {
2   font-family: Arial;
3   padding: 20px;
4 }
5 button {
6   padding: 10px;
7   margin: 5px;
8 }
9 #status {
10   margin-top: 15px;
11   font-weight: bold;
12 }
```

JS

```
1 let statusText = document.getElementById("status");
2 let successBtn = document.getElementById("successBtn");
3 let errorBtn = document.getElementById("errorBtn");
4 // funcție care creează o Promise
5 function operatieAsync(succes) {
6   return new Promise((resolve, reject) => {
7     // PENDING
8     statusText.textContent = "Status: Pending...";
9     setTimeout(() => {
10      if (succes) {
11        // FULFILLED
12        resolve("Date încărcate cu succes!");
13      } else {
14        // REJECTED
15        reject("A apărut o eroare!");
16      }
17    }, 2000);
18  });
19 // BUTON SUCCES
20 successBtn.addEventListener("click", () => {
21   operatieAsync(true)
22     .then(rez => {
23       statusText.textContent = "Fulfilled: " + rez;
24     })
25     .catch(err => {
26       statusText.textContent = "Rejected: " + err;
27     });
28 // BUTON EROARE
29 errorBtn.addEventListener("click", () => {
30   operatieAsync(false)
31     .then(rez => {
32       statusText.textContent = "Fulfilled: " + rez;
33     })
34     .catch(err => {
35       statusText.textContent = "Rejected: " + err;
36     });
37 });
```

Exemplu Promise

Succes

Eroare

Status: -

Exemplu Promise

Succes

Eroare

Fulfilled: Date încărcate cu succes!

Exemplu Promise

Succes

Eroare

Status: Pending...

Exemplu Promise

Succes

Eroare

Rejected: A apărut o eroare!

Fetch API

Ce este Fetch?

O interfață modernă pentru cereri HTTP.

Avantaje:

- Promises
- Sintaxă simplă
- Suport pentru JSON

Exemplu:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
```

```
.then(r => r.json())
```

```
.then(data => console.log(data));
```

Rezultat:

Obiect JSON cu datele utilizatorului.



1. Using Fetch API:

Clean and promise-based, perfect for most modern projects.



HTML

```
1 <html lang="ro">
2 <head>
3   <meta charset="UTF-8">
4   <title>Fetch API</title>
5   <link rel="stylesheet" href="style.css">
6 </head>
7 <body>
8 <h2>Fetch API - Exemple</h2>
9 <button id="btn">Încarcă utilizatori</button>
10 <ul id="lista"></ul>
11 <script src="script.js"></script>
12 </body>
13 </html>
```

* CSS

```
1 body {
2   font-family: Arial;
3   padding: 10px;
4 }
5 button {
6   padding: 10px;
7   margin-bottom: 15px;
8 }
9 li {
10   margin: 7px 0;
11 }
```

JS

```
1 let btn = document.getElementById("btn");
2 let lista = document.getElementById("lista");
3 btn.addEventListener("click", () => {
4   // golim lista
5   lista.innerHTML = "";
6   // FETCH API
7   fetch("https://jsonplaceholder.typicode.com/users")
8     .then(response => {
9       // transformăm răspunsul în JSON
10      return response.json();
11    })
12     .then(data => {
13       // afișăm utilizatorii
14       data.forEach(user => {
15         let li = document.createElement("li");
16         li.textContent = user.name;
17         lista.appendChild(li);
18       });
19     })
20     .catch(error => {
21       console.log("Eroare:", error);
22     });
23 });
```

Fetch API - Exemple

Încarcă utilizatori

- Leanne Graham
- Ervin Howell
- Clementine Bauch
- Patricia Lebsack
- Chelsey Dietrich
- Mrs. Dennis Schulist
- Kurtis Weissnat
- Nicholas Runolfsdottir V
- Glenna Reichert
- Clementina DuBuque

Manipularea formularelor

De ce e importantă validarea?

- Previne erorile
- Îmbunătățește experiența utilizatorului
- Protejează aplicația

Exemplu:

```
form.addEventListener("submit", e => {  
    e.preventDefault();  
    if (input.value.length < 3) {  
        alert("Minim 3 caractere!");  
    }  
});
```

Javascript Set Form Action Example

Name :

Email :

Contact No. :

Submit

Note : Form action will be set to **success.php** on click of submit button.

HTML

```
1 <html lang="ro">
2 <head>
3   <meta charset="UTF-8">
4   <title>Exemplu Formular</title>
5   <link rel="stylesheet" href="style.css">
6 </head><body>
7 <div class="container">
8   <h2>Înregistrare</h2>
9   <form id="form">
10     <input type="text" id="nume" placeholder="Nume">
11     <input type="email" id="email" placeholder="Email">
12     <input type="password" id="parola" placeholder="Parolă">
13   <button type="submit">Trimite</button>
14 </form>
15   <p id="mesaj"></p>
16 </div>
17 <script src="script.js"></script>
18 </body>
19 </html>
```

CSS

```
1 body {
2   font-family: Arial;
3   background: linear-gradient(to right, #74ebd5, #ACB6E5);
4   display: flex;
5   justify-content: center;
6   align-items: center;
7   height: 100vh;
8 }
9 .container {
10   background: white;
11   padding: 30px;
12   border-radius: 10px;
13   width: 300px;
14   text-align: center;
15   transition: 0.3s;
16 }
17 input {
18   width: 100%;
19   padding: 10px;
20   margin: 10px 0;
21   border: 2px solid #ccc;
22   border-radius: 5px;
23   transition: 0.3s;
24 }
25 input:focus {
26   border-color: #4CAF50;
27   transform: scale(1.05);
28 }
29 button {
30   padding: 10px;
31   width: 100%;
32   background: #4CAF50;
33   color: white;
34   border: none;
35   border-radius: 5px;
36   cursor: pointer;
37   transition: 0.3s;
38 }
39 button:hover {
40   background: #45a049;
41 }
42 .error {
43   border-color: red;
44 }
45 .success {
46   border-color: green;
47 }
48 #mesaj {
49   margin-top: 10px;
50   font-weight: bold;
51 }
```


JS

```
1 let form = document.getElementById("form");
2 let nume = document.getElementById("nume");
3 let email = document.getElementById("email");
4 let parola = document.getElementById("parola");
5 let mesaj = document.getElementById("mesaj");
6 // validare live
7 [nume, email, parola].forEach(input => {
8   input.addEventListener("input", () => {
9     if (input.value.length > 2) {
10      input.classList.remove("error");
11      input.classList.add("success");
12    } else {
13      input.classList.remove("success");
14    }
15  });
16 });
17 form.addEventListener("submit", (e) => {
18   e.preventDefault();
19   let valid = true;
20   // validare simplă
21   if (nume.value.length < 3) {
22     nume.classList.add("error", "shake");
23     valid = false;
24   }
25   if (!email.value.includes("@")) {
26     email.classList.add("error", "shake");
27     valid = false;
28   }
29   if (parola.value.length < 6) {
30     parola.classList.add("error", "shake");
31     valid = false;
32   }
33   // scoate shake după animație
34   setTimeout(() => {
35     document.querySelectorAll(".shake").forEach(el => {
36       el.classList.remove("shake");
37     });
38   }, 300);
39   // rezultat
40   if (valid) {
41     mesaj.textContent = "✓ Înregistrare reușită!";
42     mesaj.style.color = "green";
43     form.reset();
44     document.querySelectorAll("input").forEach(i => {
45       i.classList.remove("success");
46     });
47   } else {
48     mesaj.textContent = "✗ Completează corect!";
49     mesaj.style.color = "red";
50   }
51 });
```

Înregistrare

Înregistrare

Înregistrare

✗ Completează corect!

Înregistrare

✓ Înregistrare reușită!

Aplicații propuse spre rezolvare:

1. Creează o pagină cu: un titlu, un paragraf, un buton. La apăsarea butonului, textul paragrafului trebuie să se schimbe în: „Text actualizat din JavaScript!”.

Indicație: Folosește - `getElementById`, `textContent`, `addEventListener("click")`

2. Creează un pătrat (div) de 150x150 px. Adaugă un buton care schimbă culoarea pătratului între roșu și albastru.

Indicație: Folosește- `element.style.backgroundColor`, operatorul ternar.

3. Creează un input text și un paragraf. Pe măsură ce utilizatorul tastează, paragraful trebuie să afișeze textul introdus în timp real.

Indicație: Folosește - `input.addEventListener("input", ...)`, `input.value`.

4. Creează un formular cu câmpurile: Nume, disciplină, notă, observații. Dacă are mai puțin de 3 caractere → afișează mesaj de eroare, dacă observația are mai puțin de 5 caractere → afișează mesaj de avertizare, altfel → afișează „Formular trimis!”

Indicație: Folosește- `event.preventDefault()`, `alert()`.

5. Ai un array de numere: [1, 5, 8, 12, 3, 7], Afișează într-un paragraf: doar numerele pare, dublul fiecărui număr , suma tuturor numerelor.

Indicație: Folosește - filter(), map(), reduce()

6. Creează un obiect student cu: nume, vârstă, metodă salut() care returnează: „Salut, sunt <nume> și am <vârstă> ani.”

Afișează rezultatul metodei într-un paragraf.

7. Creează un buton „Încarcă date”. La click: afișează „Se încarcă...” , după 6 secunde, afișează „Date încărcate cu succes!” (Promises)

8. Creează un counter cu 3 butoane: + crește numărul, - scade numărul, Reset îl resetează la 0. Afișează numărul într-un element mare, centrat.

